



Member-only story

Dynamic Ontology in Practice: An Agentic, Schema-First Approach to Oncology Knowledge Graphs

A Technical Guide to Schema-First Extraction + Concept Normalization



Akash Goyal

Follow

7 min read · 4 days ago



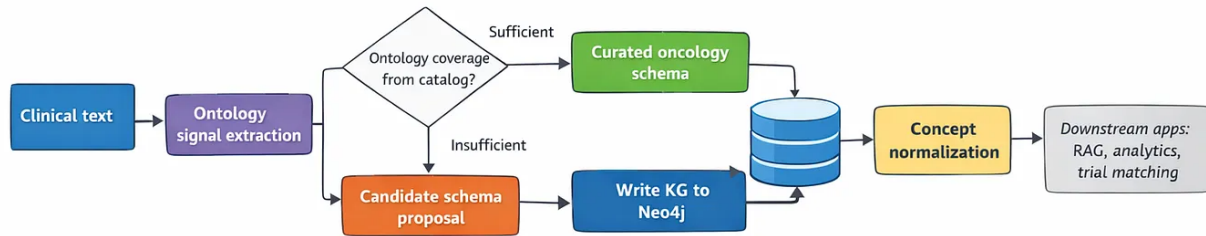
79



1



Dynamic Oncology Knowledge Graph Extraction Pipeline



“For non-subscribers: Use the [Medium friend link](#) to read.”

Most systems that extract structured data from text fail the same way: they overfit to a fixed schema or a single ontology. Reality then breaks them through synonyms, abbreviations, changing terminology, and inconsistent phrasing. This article describes a pragmatic architecture that handles this variability by splitting the problem into two layers: a small, curated schema the system can reliably extract into, and a large ontology catalog used for semantic matching and normalization.

Oncology is used as the domain because it's a strong stress test. It combines short tokens that must be exact (gene symbols), long-tail

vocabulary (drug names, regimens, biomarkers), and constant change. The approach itself is domain-agnostic.

The core design: stable schema + flexible ontology catalog

Reference ontologies (like NCI or HGNC) are great for standard IDs and synonyms, but they aren't good application schemas. They're large, complex, and built for representing knowledge — not for efficient day-to-day queries.

A curated “business ontology” is the opposite: small, simple, and opinionated. It models only the entities and relationships that actually drive value. Once your data extraction is reliable, you link those entities to reference ontologies through normalization, keeping interoperability without inheriting complexity.

This separation makes the pipeline dynamic: you can handle new or changing terminology without constantly redesigning your application's data model.

Architecture at a glance

The pipeline has four conceptual stages: (1) extract a lightweight “schema signal” from text, (2) use a Neo4j ontology catalog to estimate coverage via vector similarity, (3) extract a knowledge graph into a curated schema, and (4) normalize selected nodes by mapping them to ontology terms.

A key technical point is that “coverage” is a governance signal, not a hard dependency. Even when the catalog coverage is low, the curated schema remains stable and extraction can proceed; the system just reports lower confidence and can route to a review loop.

Ontology catalog in Neo4j as a semantic dictionary

Instead of hardcoding ontology logic, the pipeline treats ontologies as data. Terms are imported into Neo4j as `:Resource` nodes with fields like `uri`, `label`, `comment` (definition), and `prov` (provenance such as NCIt or HGNC). The important engineering decision is to compute embeddings for `label + definition` and store them as a vector property (for example, `embedding`).

Once you do that, the “what ontology term is closest to this phrase?” problem becomes a fast vector index query. That makes normalization cheap enough to do online and makes coverage measurable.

Coverage as a quantitative routing decision

Coverage answers a concrete question: given the categories implied by an input text, can the catalog find high-confidence semantic matches? This is computed by embedding the category strings, querying the vector index, and measuring the fraction that clear a similarity threshold.

In production, coverage is useful because it gives you a way to detect drift. If your system’s incoming text changes (new biomarker naming conventions, new drugs, new shorthand), you will see it as a drop in coverage or a shift in match scores. That becomes an operational signal to refresh ontologies, tune prompts, or adjust schema rules.

Extraction is constrained graph construction, not just NER

From a systems point of view, extraction is more valuable when it creates a graph, not just a list of text spans. A graph makes relationships explicit: this variant belongs to that tumor, this side effect is caused by that therapy, this treatment is given in the context of that diagnosis. Even if some links are missing or uncertain, a typed graph gives downstream systems a clear, consistent structure to rely on.

A curated schema also lowers the risk of errors. By limiting the allowed node types and relationships, the extractor has fewer choices to make, which reduces hallucinated connections and inconsistent labeling

Normalization: grounding nodes to URIs without losing raw values

Normalization is where the flexible ontology catalog becomes valuable. Instead of trying to fully encode the ontology structure in your application graph, the pipeline stores **grounding metadata** on extracted nodes.

The principle is: keep the raw extracted value for traceability, and store the normalized mapping alongside it so downstream systems can choose their own trust policy.

A typical normalized node stores fields like:

- `normalized_uri` : canonical identifier for joins and dedup
- `normalized_label` : human-friendly name for UX
- `normalized_prov` : source ontology (NCIt, HGNC)
- `normalized_score` : similarity score for thresholding
- `normalized_method` : e.g., `vector` (or later, hybrid)

This approach makes normalization auditable and reversible.

Why gene normalization is separated

Gene symbols are short and ambiguous. Even with a good ontology, doing generic vector search across all biomedical concepts can return results that look reasonable but are wrong when the input is only a 3–5 character token.

A practical solution is to normalize genes in a second pass limited to HGNC. This restricts candidates to gene-like entities, which sharply reduces false matches. From an engineering perspective, it also improves observability: gene normalization failures are easier to detect, debug, and fix.

A concrete example (technical)

Consider an input string that mentions a cancer type, a variant, a biomarker, and a drug:

“Metastatic NSCLC. EGFR L858R. PD-L1 60%. Started osimertinib.”

The extracted application graph would typically include nodes such as `CancerDiagnosis`, `Tumor`, `GenomicVariant`, `Biomarker`, and `Drug`, plus edges such as `HAS_TUMOR`, `HAS_VARIANT`, `HAS_BIOMARKER`, and `TREATED_WITH`.

Normalization then grounds `Drug(name=osimertinib)` to an NCIt URI, and grounds `GenomicVariant(gene=EGFR)` to an HGNC URI.

Downstream you can join by URIs even if raw spellings differ.

How normalization queries look conceptually (Cypher snippets)

The following snippet illustrates the core pattern: compute an embedding for an extracted string and query the Neo4j vector index to retrieve the best-matching `:Resource`.

```
CALL db.index.vector.queryNodes(
  $vectorIndex,
  1,
  genai.vector.encode($text, 'OpenAI', { token: $openaiToken })
) YIELD node, score
RETURN node.uri, node.label, node.prov, score;
```

Once you have the best match, you attach it to the extracted node as properties.

```
MATCH (d:Drug)
WHERE d.name IS NOT NULL AND d.normalized_uri IS NULL
WITH d, d.name AS q
CALL db.index.vector.queryNodes($vectorIndex, 1, genai.vector.encode(q,
YIELD node AS r, score
WHERE score >= $threshold AND r.prov = 'NCIt'
SET d.normalized_uri = r.uri,
    d.normalized_label = r.label,
    d.normalized_prov = r.prov,
    d.normalized_score = score,
    d.normalized_method = 'vector';
```

These snippets are intentionally minimal. In real systems, you batch

updates, record multiple candidates for review, and may use hybrid scoring (lexical + vector) for short tokens.

Interaction flow (systems perspective)

The pipeline is easiest to reason about when expressed as a sequence of calls between components: text source, schema-signal extractor, ontology catalog, KG extractor, and graph store.

This makes it clear where you can add caching (ontology lookups), where you can add review gates (low coverage), and where you can add auditing (store match candidates and scores).

A minimal conceptual data model

A good schema is a contract. You want stable labels and stable relationship types because they become the public API for downstream queries. The reference ontology is then “linked” via properties that store URIs.

This design is intentionally “lightweight” in terms of ontology structure. It preserves interoperability through URIs, while keeping the operational model simple.

Reliability and failure modes (engineering view)

A common mistake is trusting normalization too much. Similarity scores don't prove correctness, they only rank candidates. Production systems usually add safeguards like score thresholds, provenance

filters, and stricter rules for short tokens, such as exact matches, curated synonym lists, or HGNC-only matching.

Another frequent issue is schema sprawl. As a curated schema grows without discipline, extraction quality drops because the model has too many options. Keeping the schema small and evolving it deliberately is almost always the better tradeoff.

The third failure mode is silent drift. If the ontology catalog isn't kept up to date, new drugs or biomarker naming patterns slowly reduce coverage and match quality. That's why the pipeline tracks coverage and match scores: to make this drift visible instead of letting it go unnoticed.

Conclusion

A schema-first extraction approach gives you a stable contract. A Neo4j ontology catalog with embeddings gives you a pragmatic normalization mechanism. The combination provides an end-to-end path from text to queryable graph without requiring you to hardcode every synonym, while still keeping a trail of scores and provenance for audit.

Once you have established this foundation, you can add higher-level capabilities, such as candidate curation workflows, hybrid normalization (lexical and vector), ontology version pinning, and automated regression tests on extraction/normalization quality, without altering the core architecture.

Neo4j

Schema Design

Ontology

Entity Resolution

System Design Concepts



Written by Akash Goyal

Follow

893 followers · 216 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning <https://www.linkedin.com/in/akash-g-7a5224246/>

More from Akash Goyal

 Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind...

A Cost-Optimized, Fully Observable GraphRAG Operating System Built for...

★ Dec 14, 2025 🖱 593 💬 6 📌+ ...

 Akash Goyal

Why LLMs Keep Forgetting Ontologies (And How to Fix It)

Production Architecture for Ontology Memory (Neo4j + LangGraph)

★ Dec 23, 2025 🖱 238 💬 4 📌+ ...

 Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval...

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform...

★ Dec 13, 2025 🖱 265 💬 4 📌+ ...

 Akash Goyal

A Complete Guide to Agentic Context Graphs: Turning...

Why capturing the “why” behind decisions is the next great leap in...

★ Jan 10 🖱 149 💬 2 📌+ ...

See all from Akash Goyal

Recommended from Medium



In Neo4j Developer Blog by William Lyon

Hands On With Context Graphs And Neo4j

Building AI systems that don't just make decisions, but remember and use the...

5d ago

164

1



Pankaj Kumar

From Protégé to Production: Integrating Your Ontology wit...

Your Ontology is Built. Now What?



Jan 10

43



 Fanghua (Joshua) Yu

A Unified Framework for AI-Native Knowledge Graphs

Part 1—Generative Knowledge Modeling

4d ago  23  1



 Vishal Mysore

Agentic Knowledge Graphs with A2UI: Why AI Reasoning...

Agentic Knowledge Graphs represent a new class of AI-driven visualization...

5d ago  70  1



 In Data Science Collective by Shenggang Li

Why Building AI Agents Is Mostly a Waste of Time

The Structural, Mathematical, and Economic Limits of RAG Pipelines

★ 6d ago  757  44



 Akash Goyal

A Complete Guide to Agentic Context Graphs: Turning...

Why capturing the “why” behind decisions is the next great leap in...

★ Jan 10  149  2



See more recommendations

